

Machine Learning Applications in Physics

PHYG004 — 2026 Spring

Prof. Young Woo Choi

Department of Physics, Sogang University

Wed & Fri, 09:00 – 10:15

6/5/2026

Instructor

Name	Young Woo Choi (최영우)
Email	<u>ywchoi02@sogang.ac.kr</u>
Office	F303C
Office Hours	Mon, Tue, Thu 10:00–12:00 (by appointment)
Homepage	<u>https://yw-choi.github.io</u>

Please make an appointment by email before visiting.

Course Overview

- ML from a physicist's perspective: how neural networks can study quantum, statistical, and condensed matter systems
- Topics: basic neural networks, transformers, symmetry-aware models, generative models, differentiable physics, simulation-based inference
- Course format: 50% lectures + 50% student presentations
- No single textbook — lecture notes, coding examples, and selected papers provided throughout
- Hands-on coding with JAX in every lecture

Course Objectives

1. Understand the basic concepts and architectures of deep learning from a physics perspective
2. Learn how deep learning models are applied to problems in quantum, statistical, and condensed matter physics
3. Gain experience formulating physics problems as machine learning tasks and implementing solutions using JAX
4. *Learn how to effectively use AI agents for programming and best practices for AI-assisted development*

Grading

Component	Weight	Details
Individual Presentation	30%	Week 7, 3-minute oral presentation
Term Project	50%	Team-based (up to 3), presented in Week 15
Participation	20%	Attendance, in-class engagement

- No midterm or final exams

Course Schedule: Foundations Block (L01–L10)

Lecture	Topic	Data
L01 (03/04)	JAX, Autodiff & Physics-vs-ML	toy/synthetic
L02 (03/07)	Why ML for Physics: Approximation, Inference, Surrogate	survey
L03 (03/11)	Neural Network Foundations: Backprop, Activations	toy/synthetic
L04 (03/14)	Classification & Information Theory	Fashion-MNIST (real)
L05 (03/18)	MLP Deep Dive: Optimization, Overfitting, Init	toy/synthetic
L06 (03/21)	Convolutional Networks: LeNet to Phase Classification	toy/synthetic
L07 (03/25)	CNN Physics Applications: Real Data & Transfer Learning	Galaxy10 (real)
L08 (03/28)	Automatic Differentiation & PINNs (Full Session)	toy/synthetic

Course Schedule: Symmetry, Generative & Inference Blocks (L11–L20)

Lecture	Topic	Data
L11 (04/29)	Graph Neural Networks & Geometric DL	QM9 (real)
L12 (05/06)	E(3)-Equivariant Networks: Irreps & Tensor Products	rMD17 (real)
L13 (05/13)	ML Potentials & Foundation-Model Fine-Tuning	Materials Project (real)
L14 (05/20)	Variational Autoencoders: ELBO as Variational Free Energy	mixed
L15 (05/27)	Normalizing Flows: Change of Variables & Boltzmann Generators	mixed
L16 (06/03)	Flow Matching: Velocity Fields & Simulation-Free CNFs	mixed
L17 (06/05)	Diffusion Models (DDPM): Forward/Reverse & Score/SDE	mixed

Special Weeks

Week	Event
Week 7 (04/15)	Individual Presentations
Week 8 (04/22)	Midterm Week — no exam; L10 transformer + project prep
Week 15 (06/10)	Term Project Presentations
Week 16 (06/17)	Final Exam Week — no exam; project wrap-up & course retrospective

Individual Presentation (Week 7)

- **3-minute** oral presentation (slides optional)
- **Goal:** Propose an ML approach to a physics problem you care about

Suggested structure

1. **Problem** — What physics question are you trying to answer?
 2. **Why ML?** — Why is this hard with traditional methods?
 3. **Approach** — What type of model / data would you use?
 4. **Expected outcome** — What would success look like?
-
- No code or implementation needed — this is about **ideas**
 - Can draw from your own research, coursework, or a paper you've read
 - This presentation can serve as a starting point for your term project

Term Project (Week 15)

- **Team-based:** up to 3 students per team
- Apply machine learning to a physics problem:
 - Reproduce results from a research paper, **or**
 - Original exploratory study
- Final presentation in Week 15
- More details will be provided later in the semester

Registration (Shared Google Sheet)

- Register your **GitHub ID** to the shared sheet (due today)
- Register your topic for **individual presentation** (due Week 6)
- **Team formation** for term project (due Week 7)

Policy on AI Usage

- Use of AI tools (ChatGPT, Copilot, Claude, etc.) is **strongly encouraged**
- Practicing with AI agents (VS Code + Copilot Agents) is a key part of this course
- However, you must **always understand** the output AI generates for you
- Always ask yourself: **how would I verify this result?**

AI amplifies your ability. $0 \times 100 = 0$

Development Environment

Option A: VS Code + GitHub Copilot (Recommended)

- Full-featured IDE with AI-assisted coding
- Free for students via GitHub Education Pack
- Works with local Python / JAX installation

Option B: Google Colab

- No local setup required
- Free GPU access (limited)
- Great for quick experimentation

Bring your laptop to class — we'll do hands-on exercises!

GitHub Education Pack

Get **free GitHub Copilot and Copilot Chat** access as a student:

1. Go to education.github.com
2. Click "**Join GitHub Education**"
3. Verify your student status with your Sogang email
4. Once approved, activate **GitHub Copilot** in your account settings
5. Install the **GitHub Copilot extension** in VS Code

This gives you access to **Copilot Agents** — AI that can write, run, and debug code for you!

Course Tools

Tool	Purpose
JAX	Primary ML framework (autodiff, JIT, GPU)
NumPy	Array operations, data preparation
Matplotlib	Visualization and plotting
Equinox	Neural network library for JAX
Optax	Gradient-based optimization

All tools are open-source and free. We'll set everything up today!

Data Label Policy

Every lecture intro identifies the **data type** used in that session:

Label	Meaning
toy/synthetic	Analytic functions, MC-generated samples, no real measurement
real data	Actual experimental or DFT-computed datasets (QM9, Galaxy10, rMD17, ...)
survey	Conceptual/motivational — no dataset required
mixed	Both toy demonstrations and real-data experiments

This lecture: toy/synthetic — analytic potentials (Lennard-Jones, harmonic oscillator) and Ising Monte Carlo generated on the fly.

Why JAX?

- **Functional programming** style — natural for physicists (think: mathematical functions)
- **Automatic differentiation** — `jax.grad` differentiates any Python function
- **JIT compilation** — `jax.jit` compiles functions for GPU/TPU speed
- **Vectorization** — `jax.vmap` auto-vectorizes over batch dimensions
- **NumPy-compatible** API — easy to learn if you know NumPy
- Widely used in physics ML research

$$\text{Force} = -\nabla V \quad \longrightarrow \quad \text{force} = -\text{jax.grad}(V)$$

JAX: Four Core Transformations

Transformation	Physics analogy	Use in this course
<code>jax.grad</code>	$-\nabla V, \partial \mathcal{L} / \partial \theta$	Autodiff forces, backprop
<code>jax.jit</code>	Compile once, run many	Fast training loops
<code>jax.vmap</code>	Batch over ensemble	Vectorized MC sweeps
<code>jax.value_and_grad</code>	V and ∇V together	Efficient loss + gradient

These four cover 90% of what you need. We introduce each one with a physics example today.

JAX vs NumPy: Key Differences

```
# NumPy - mutable, familiar
import numpy as np
a = np.array([1.0, 2.0, 3.0])
a[0] = 99.0          # in-place mutation OK

# JAX - immutable, functional
import jax.numpy as jnp
a = jnp.array([1.0, 2.0, 3.0])
a = a.at[0].set(99.0) # returns new array; a is unchanged
```

Why immutability?

- Enables **pure functions** → safe to differentiate and JIT-compile
- No hidden side effects — same input always gives same output
- Think of it as **referential transparency** (math functions)

Explicit PRNG in JAX

NumPy random state is global and mutable. JAX makes randomness **explicit and reproducible**:

```
import jax
import jax.random as random

key = random.PRNGKey(42)          # deterministic seed

key1, key2 = random.split(key)    # split into two independent keys
x = random.normal(key1, (100,))  # samples from key1
y = random.normal(key2, (100,))  # independent samples from key2
```

Always split keys before use. Never reuse a key — results may correlate unexpectedly.

This pattern lets you reproduce any stochastic computation exactly by controlling the key chain.

Automatic Differentiation: The Physics View

Physics is full of derivative computations:

$$F = -\frac{dV}{dr}, \quad \nabla_{\theta} \mathcal{L}, \quad \frac{\partial^2 \psi}{\partial x^2}$$

JAX computes **exact** derivatives of any Python function via **reverse-mode AD** (backpropagation) or **forward-mode AD**:

```
def V_harmonic(x, k=1.0):  
    return 0.5 * k * x**2  
  
force = jax.grad(V_harmonic)    # dV/dx → -force  
print(force(2.0))              # -2.0 (exact, not finite difference)
```

- No symbolic manipulation, no finite-difference approximation
- Works through **any** differentiable Python: loops, conditionals, recursion
- Composes with `jit` and `vmap` freely

Autodiff vs Finite Differences

Method	Cost	Accuracy	Scales to N params
Finite difference	$\mathcal{O}(N)$ evaluations	$\mathcal{O}(h)$ truncation error	No
Forward-mode AD	$\mathcal{O}(N)$ passes	Exact (machine precision)	Expensive for large N
Reverse-mode AD (backprop)	$\mathcal{O}(1)$ passes	Exact	Yes

Reverse-mode AD makes gradient-based optimization of millions of parameters feasible. This is why deep learning works at scale.

Backprop $\equiv$ Reverse-mode AD on the computation graph

JIT Compilation

`jax.jit` traces your function once and compiles it to XLA (Accelerated Linear Algebra):

```
@jax.jit
def update(params, x, y):
    loss, grads = jax.value_and_grad(mse_loss)(params, x, y)
    return loss, jax.tree.map(lambda p, g: p - 0.01 * g, params, grads)
```

- **First call:** tracing + compilation (slow)
- **Subsequent calls:** compiled binary (fast, GPU/TPU ready)
- Constraint: shapes and dtypes must be **static** at trace time

Rule of thumb: JIT your outer training loop, not individual array operations.

vmap: Vectorization Without Loops

`jax.vmap` lifts a function that operates on a single example to operate on a batch:

```
def harmonic_energy(x, k):  
    return 0.5 * k * x**2  
  
# Compute energy for a batch of positions and spring constants  
batch_energy = jax.vmap(harmonic_energy)  
xs = jnp.array([1.0, 2.0, 3.0])  
ks = jnp.array([0.5, 1.0, 2.0])  
print(batch_energy(xs, ks))  # [0.25, 2.0, 9.0]
```

- No explicit Python loop — the vectorization happens inside XLA
- Composes with `jit`: `jax.jit(jax.vmap(f))` is the standard pattern
- Physics use: batch over ensemble members, MC walkers, particles

Today's Hands-On: Unified Notebook

Data label: toy/synthetic (analytic potentials + Ising MC)

The single notebook (`handson.ipynb`) covers four topics in order:

1. **JAX functional style** — `jax.grad` on Lennard-Jones potential
2. **Explicit PRNG** — `random.split` for Ising lattice initialization
3. **vmap** — batch curvature of harmonic potential
4. **jit** — speed comparison on QHO Monte Carlo warm-up

Each section ends with a **checkpoint exercise** that has a physics-grounded checksum (e.g., $r = 2^{1/6}\sigma$ is the LJ equilibrium distance where $F = 0$).

Benchmark Systems: Named Spine

Two **named benchmark systems** appear throughout the course. You will see them again and again — each time in a new ML context.

Ising Model — $H = -J \sum_{\langle i,j \rangle} s_i s_j$

Introduced today (L01) as a JAX hands-on target. Returns in L06 (CNN phase classification), L07 (real-data CNN), L14 (VAE latent magnetization).

Boltzmann Double-Well — $V(x) = (x^2 - 1)^2$

Introduced in L02 as a gradient descent target. Returns in L15 (normalizing flow Boltzmann generator), L16 (flow matching velocity field), L17 (diffusion reverse process).

Why Two Benchmark Systems?

These systems are chosen deliberately:

Property	Ising	Double-well
Physics	Statistical mechanics, phase transition	Energy landscape, metastability
ML challenge	High-dimensional state space	Multimodal distribution
Grows with the course	Toy → real-data → latent space	1D toy → Boltzmann generator → molecular

When you encounter a new method (VAE, normalizing flow, diffusion), ask yourself: **How does this method handle the Ising or double-well problem differently from the previous method?**

Physics Analogies: A Recurring Dictionary

ML concept	Physics analogue
Loss function $\mathcal{L}$	Free energy / Action
Gradient descent	Energy minimization
Regularization	Prior / Constraint
Latent variable z	Collective coordinate / Order parameter
Equivariance	Symmetry of physical laws
Sampling (generative model)	Boltzmann / Langevin sampling

Today: What to Expect

In the hands-on tutorial, we will:

1. **Install** JAX and verify GPU/CPU backend
2. **JAX functional style** — immutability, pure functions, tree utilities
3. **Autodiff** — `jax.grad` on Lennard-Jones; compare with finite differences
4. **Explicit PRNG** — `random.split` for reproducible Ising initialization
5. **vmap** — batch harmonic curvature in one line
6. **jit** — compile and benchmark QHO Monte Carlo warm-up
7. **Ising MC** — single-temperature run, magnetization output
8. **Named Spine** — Ising and double-well declared as semester benchmarks

Open the notebook on Google Colab or your local environment.

Checkpoint Exercises

Exercise 1 — `jax.grad` on Lennard-Jones:

$$V(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right]$$

Compute $F = -dV/dr$ with `jax.grad` and compare to finite difference.

Checksum: $F = 0$ at $r = 2^{1/6}\sigma$ (equilibrium distance).

Exercise 2 — PRNG pattern:

Use `random.split` to initialize three $L = 8$ Ising lattices with independent keys. Verify that reusing the same key produces identical lattices.

Summary

Today you learned:

- JAX's four core transformations: `grad`, `jit`, `vmap`, `value_and_grad`
- Why immutability and explicit PRNG matter for reproducible physics computation
- Reverse-mode AD gives exact gradients at $\mathcal{O}(1)$ backward passes — the engine of all ML training
- Two benchmark systems — Ising and double-well — will reappear as the semester progresses

Next lecture (L02): Why ML for Physics? — three families of physics problems (approximation, inference, surrogate modeling), and a first look at optimization on the double-well energy landscape.